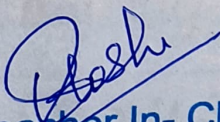


Thadomal Shahani Engineering College

Bandra (W.), Mumbai - 400 050.

❧ CERTIFICATE ❧

Certify that Mr./Miss Aarya Vijay Arban
of IT Department, Semester III with
Roll No. 07 has completed a course of the necessary
experiments in the subject DSA under my
supervision in the **Thadomal Shahani Engineering College**
Laboratory in the year 2023 - 2024


Teacher In- Charge

Head of the Department

Date 25/10/23

Principal

Assignment - 1

Aim: Implementing Linear Data structures using Arrays and performing insertion, deletion, copying an array to other, searching and sorting

Diagram for **Insertion** of an element in an array:

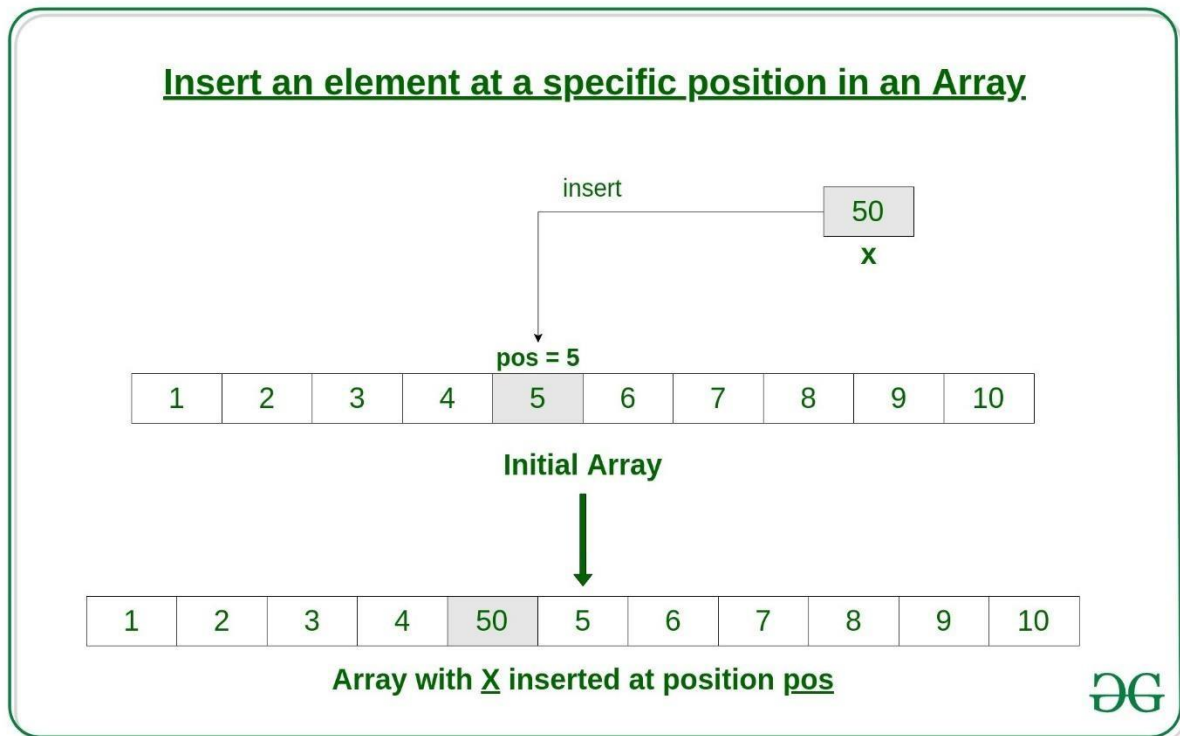


Diagram for **Deletion** of an element from an array:

Delete Operation in Unsorted Array

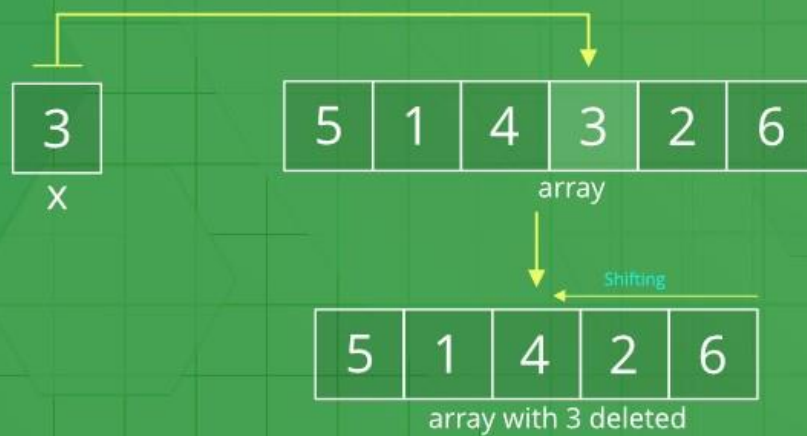


Diagram for implementing **Bubble Sort Algorithm**:

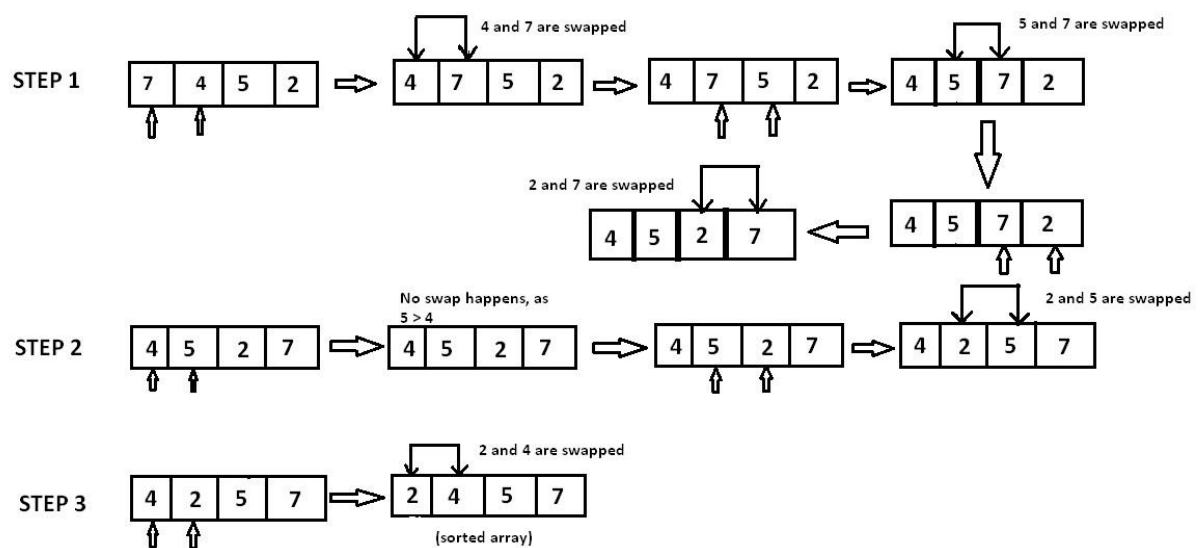
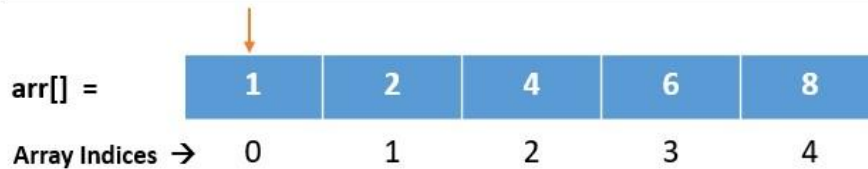


Diagram for implementing **Linear Search Algorithm**:

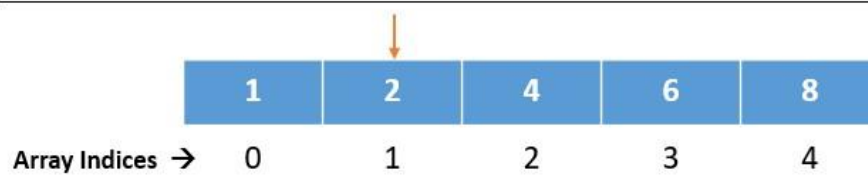
Linear Search Algorithm

Element to search for: 4

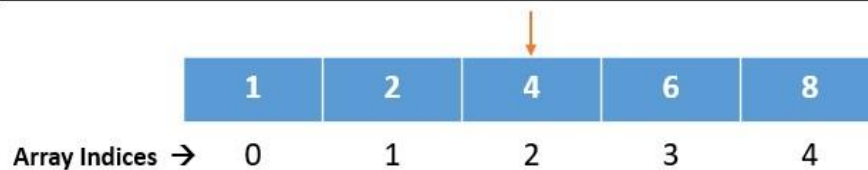
Iteration 1: Begin search from index 0 and check $\text{arr}[0] == \text{element}$



Iteration 2: Move to next element and check $\text{arr}[1] == \text{element}$



Iteration 3: Move to next element and check $\text{arr}[2] == \text{element}$



Element Found!

Program:

```
#include<stdio.h>
void main()
{
    int n,m,p,i,j,a[10],b[10],c[10],loc,z,temp,x,found=0;
    //Creating an array
    printf(" Enter the number of elements: ");
    scanf("%d",&n);
    m=n;
    p=n;
    for(i=0;i<n;i++)
    {
        printf("\n Enter number: ");
        scanf("%d",&a[i]);
    }
}
```

```

}
//Copying an array
for(i=0;i<n;i++)
{
    b[i]=a[i];    c[i]=a[i];
}
//Displaying the original array
printf("\n The original array:");
for(i=0;i<n;i++)
{
    printf("\t %d",a[i]);
}
//Displaying the copied array    printf("\n The copied array:");    for(i=0;i<n;i++)
{
    printf("\t %d",b[i]);
}
//Deletion of an element
printf("\n Enter the location of the element to be deleted: ");
scanf("%d",&loc);
while(loc<n)
{
    a[loc-1]=a[loc];
    loc++;
}
n--;
for(i=0;i<n;i++)
{
    printf("\t %d",a[i]);
}
//Insertion of an element
printf("\n Enter the element to be inserted: ");
scanf("%d",&z);
printf("\n Enter the location of the element to be inserted: ");
scanf("%d",&loc);
for(i=m-1;i>=loc-1;i--)
{
    b[i+1]=b[i];
}
b[loc-1]=z;
for(i=0;i<m+1;i++)
{
    printf("\t %d",b[i]);
}
//Bubble Sort
for(i=0;i<p;i++)

```

```

{
    for(j=0;j<p-i;j++)
    {
        if(c[j]>c[j+1])
        {
            temp=c[j];
            c[j]=c[j+1];
            c[j+1]=temp;
        }
    }
}
printf("\n Sorted Array: \n");
for(i=0;i<p;i++)
{
    printf("\t %d",c[i]);
}
//Linear Search
printf("\n Enter element to be searched: ");
scanf("%d",&x);
for(i=0;i<p;i++)
{
    if(x==c[i])
    {
        printf("\n Element is present");
        found=1;
    }
}
if(!found)
{
    printf("\n Element is not present");
}
}

```

Output:

Enter the number of elements: 4

Enter number: 1

Enter number: 3

Enter number: 2

Enter number: 4

The original array: 1 3 2 4

The copied array: 1 3 2 4

Enter the location of the element to be deleted: 2

1 2 4

Enter the element to be inserted: 5

Enter the location of the element to be inserted: 3

1 3 5 2 4

Sorted Array:

1 2 3 4

Enter element to be searched: 2

Element is present

Assignment – 2

Aim: To implement Merge Sort and Quick Sort.

Code (Merge Sort):

```
#include <stdio.h>
#include <stdlib.h>
int count=0;
int size;
int main()
{
    int quit1=0, choice1;
    printf("\nEnter Size of Array: \n");
    scanf("%d", &size);
    int nums[size];
    printf("\nEnter Array Elements: \n");
    for (int i = 0; i<size; i++) {
        printf("%d. ", i+1);
        scanf("%d", &nums[i]);
    }
    while (quit1 == 0) {
        printf("\nSelect An Option from the Menu\n1. Display Entered Array\n2. Sort Entered
Array\n3. Quit\n");
        scanf("%d", &choice1);
        switch(choice1) {
            case 1:
                printf("\nEnter Array: \n");
                printArray(nums, size);
                printf("\n");
                break;
            case 2:
                mergeSort(nums, 0, size - 1);
```

```

printf("\nSorted Array: \n");
printArray(nums, size);
printf("\n");
break;

```

default:

```

quit1 = 1;
break;

```

```

}
}

```

```

return 0;

```

```

}

```

```

void printArray (int nums[], int size) {

```

```

printf("[");

```

```

for (int i = 0; i<size; i++) {

```

```

printf("%d\t", nums[i]);

```

```

}

```

```

printf("]");

```

```

}

```

```

void merge(int nums[], int firstIndex, int partition, int

```

```

lastIndex)

```

```

{

```

```

int len1, len2;

```

```

int i ,j,k;

```

```

len1 = partition - firstIndex + 1;

```

```

len2 = lastIndex - partition;

```

```

int nums1[len1], nums2[len2];

```

```

for (i = 0 ; i<len1; i++) {

```

```

nums1[i] = nums[i+firstIndex];

```

```

}

```

```

for (j = 0; j<len2; j++) {

```

```

nums2[j] = nums[j + partition+1];

```

```

}

```

```

i = 0;

```

```

j= 0;

```

```

k = firstIndex;

```

```

while (i<len1 && j<len2) {

```

```

if (nums2[j] > nums1[i]) {

```

```

nums[k] = nums1[i];

```

```

i++;

```

```

}

```

```

else {

```

```

nums[k] = nums2[j];

```

```

j++;
}
k++;
}
while (i<len1) {
nums[k] = nums1[i];
i++;
k++;
}
while (j < len2) {
nums[k] = nums2[j];
j++;
k++;
}
printf("\nIteration Number: %d\n", ++count);
printArray(nums, size);
printf("\n");
}
void mergeSort(int nums[], int firstIndex, int lastIndex)
{
if (firstIndex < lastIndex) {
int partition = firstIndex + (lastIndex - firstIndex)/2;
mergeSort(nums, firstIndex, partition);
mergeSort (nums, partition+1, lastIndex);
merge(nums,firstIndex, partition, lastIndex);
}
}

```

Output (Merge Sort):

Enter Size of Array:

4

Enter Array Elements:

1. 3

2. 4

3. 1

4. 2

Select An Option from the Menu

1. Display Entered Array

2. Sort Entered Array

3. Quit

1

Entered Array:

[3 4 1 2]

Select An Option from the Menu

1. Display Entered Array
 2. Sort Entered Array
 3. Quit
- 2

Iteration Number: 1

[3 4 1 2]

Iteration Number: 2

[3 4 1 2]

Iteration Number: 3

[1 2 3 4]

Sorted Array:

[1 2 3 4]

Select An Option from the Menu

1. Display Entered Array
 2. Sort Entered Array
 3. Quit
- 3

Process returned 0 (0x0) execution time : 18.860 s

Press any key to continue.

Code (Quick Sort):

```
#include <stdio.h>
#include <stdlib.h>
int count=0;
int main()
{
    int quit1=0,size, choice1;
    printf("\nEnter Size of Array: \n");
    scanf("%d", &size);
    int nums[size];
    printf("Enter Array Elements: \n");
    for (int i = 0; i<size; i++) {
        printf("%d. ", i+1);
```

```

scanf("%d", &nums[i]);
}
printf("\nEntered Array: \n");
printArray(nums, size);
printf("\n");
int sorted[size];
quickSort(nums, 0, size-1, size);
printf("\nSorted Array: \n");
printArray(nums, size);
printf("\n");
return 0;
}
void printArray (int nums[], int size) {
printf("[");
for (int i = 0; i<size; i++) {
printf("%d\t", nums[i]);
}
printf("]");
}
void quickSort(int nums[], int sl, int el, int size1) {
int x,y,z;
if (sl == el || sl > el) {
return;
}
count++;
int size = el-sl+1;
int leftPointer = sl;
int rightPointer = el;
int pivot = sl;
while (leftPointer != rightPointer) {
if (pivot == rightPointer) {
if (nums[pivot] < nums[leftPointer]) {
int temp = nums[pivot];
nums[pivot] = nums[leftPointer];
nums[leftPointer] = temp;
pivot = leftPointer;
} else {
leftPointer++;
}
}
continue;
}
if (pivot == leftPointer) {
if (nums[pivot] > nums[rightPointer]) {
int temp = nums[pivot];
nums[pivot] = nums[rightPointer];

```

```

nums[rightPointer] = temp;
pivot = rightPointer;
}
else {
rightPointer--;
}
continue;
}
}
printf("\nIteration No: %d\n", count);
printArray(nums, size1);
printf("\n");
quickSort(nums, sl, pivot-1, size1);
quickSort(nums, pivot+1, el, size1);
}

```

Output (Quick Sort):

Enter Size of Array:

4

Enter Array Elements:

1. 2

2. 3

3. 1

4. 4

Entered Array:

[2 3 1 4]

Iteration No: 1

[1 2 3 4]

Iteration No: 2

[1 2 3 4]

Sorted Array:

[1 2 3 4]

Assignment - 3

Aim: Write a program to implement Stack and Queue using arrays.

Code (Stack):

```
#include<stdio.h>
```

```
int stack[100], i, choice=0, top=-1,n;
```

```

void main()
{
    printf("Enter the number of elements in the stack \n");
    scanf("%d",&n);
    while(choice!=4)
    {
        printf("\n Choose one of the following choices:\n");
        printf("\n 1.PUSH");
        printf("\n 2.POP");
        printf("\n 3.SHOW");
        printf("\n 4.EXIT");
        printf("\n Enter the choice:\n");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: show();
            break;
            case 4: printf("Exit");
            break;
            default: printf("invalid choice!!");
        }
    }
}

void push(int val, int n)
{
    if(top==n)
    {
        printf("\n Overflow");
    }
    else
    {
        printf("Enter the value:");
        scanf("%d", &val);
        top=top+1;
        stack[top]=val;
    }
}

int pop()
{
    if(top== -1)
    {

```

```
printf("\n Underflow");
return 0;
}
else
{
top=top-1;
return stack[top];
}
}
void show()
{
for(i=top;i>=0; i--)
{
printf("%d\t", stack[i]);
}
if(top== -1)
{
printf("Stack is empty");
}
}
```

Output (Stack) :

Enter the number of elements in the stack

4

Choose one of the following choices:

- 1.PUSH
- 2.POP
- 3.SHOW
- 4.EXIT

Enter the choice:

1

Enter the value:1

Choose one of the following choices:

- 1.PUSH
- 2.POP
- 3.SHOW
- 4.EXIT

Enter the choice:

1

Enter the value:2

Choose one of the following choices:

- 1.PUSH
- 2.POP
- 3.SHOW
- 4.EXIT

Enter the choice:

3

2 1

Choose one of the following choices:

1.PUSH

2.POP

3.SHOW

4.EXIT

Enter the choice:

2

Choose one of the following choices:

1.PUSH

2.POP

3.SHOW

4.EXIT

Enter the choice:

3

1

Choose one of the following choices:

1.PUSH

2.POP

3.SHOW

4.EXIT

Enter the choice:

4

Exit

Code (Queue):

```
#include <stdio.h>
```

```
int queue[100];
```

```
void main()
```

```
{
```

```
    int choice=0,size;
```

```
    printf("Enter the number of elements in queue: ");
```

```
    scanf("%d",&size);
```

```
    int front=-1,rear=-1;
```

```
    while(choice<4)
```

```
    {
```

```
        printf("\n1.Enqueue\n2.Dequeue\n3.Display\n4.Exit\nEnter your choice: ");
```

```
        scanf("%d",&choice);
```

```
        switch(choice)
```

```
        {
```

```
            case 1:
```

```
                enqueue(&front,&rear,size);
```

```
                break;
```

```

case 2:
dequeue(&front,&rear,size);
break;
case 3:
display(&front,&rear);
break;
case 4:
printf("Exiting..");
break;
default:
printf("invalid choice");
break;
}
}
}
void enqueue(int*p, int*q, int len)
{
int value;
if(*q==len-1)
{
printf("\nOverflow\n");
}
else
{
if((*p)==-1)
{
*p=0;
}
printf("\nEnter the value you want to insert to the queue: ");
scanf("%d",&value);
(*q)++;
queue[*q]=value;
}
}
void dequeue(int *p,int*q,int len)
{
if(*p==-1 || *p>*q)
{
printf("\nUnderflow\n");
}
else
{
(*p)++;
}
}

```

```

void display(int*p,int*q)
{
int i;
if(*p== -1 || *p>*q)
{
printf("Queue is empty\n");
}
else
{
for(i=*p;i<=*q;i++)
{
printf("%d\n",queue[i]);
}
}
}

```

Output (Queue) :

Enter the number of elements in queue: 4

- 1.Enqueue
- 2.Dequeue
- 3.Display
- 4.Exit

Enter your choice: 1

Enter the value you want to insert to the queue: 1

- 1.Enqueue
- 2.Dequeue
- 3.Display
- 4.Exit

Enter your choice: 1

Enter the value you want to insert to the queue: 2

- 1.Enqueue
- 2.Dequeue
- 3.Display
- 4.Exit

Enter your choice: 3

1
2

- 1.Enqueue
- 2.Dequeue
- 3.Display
- 4.Exit

Enter your choice: 2

- 1.Enqueue
- 2.Dequeue
- 3.Display

4.Exit

Enter you choice: 3

2

1.Enqueue

2.Dequeue

3.Display

4.Exit

Enter your choice: 4

Exiting..

Assignment – 4

Aim: Write a program for implementation of different operations of Singly Linked List.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node *next;
};
struct node *head = NULL;
void insert_at_beginning(int data) {
    struct node *new_node = malloc(sizeof(struct node));
    new_node->data = data;
    new_node->next = head;
    head = new_node;
}
void insert_at_end(int data) {
    struct node *new_node = malloc(sizeof(struct node));
    new_node->data = data;
    new_node->next = NULL;
    if (head == NULL) {
        head = new_node;
        return;
    }
    struct node *temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = new_node;
}
void insert_at_specified_location(int data, int location) {
```

```

if (location == 1) {
insert_at_beginning(data);
return;
}
struct node *new_node = malloc(sizeof(struct node));
new_node->data = data;
new_node->next = NULL;
struct node *temp = head;
int i = 1;
while (i < location - 1 && temp != NULL) {
temp = temp->next;
i++;
}
if (temp == NULL) {
printf("The specified location is out of bounds.\n");
return;
}
new_node->next = temp->next;
temp->next = new_node;
}
void delete_at_beginning() {
if (head == NULL) {
printf("The linked list is empty.\n");
return;
}
struct node *temp = head;
head = head->next;
free(temp);
}
void delete_at_end() {
if (head == NULL) {
printf("The linked list is empty.\n");
return;
}
struct node *temp = head;
while (temp->next->next != NULL) {
temp = temp->next;
}
struct node *to_delete = temp->next;
temp->next = NULL;
free(to_delete);
}
void delete_at_specified_location(int location) {
if (location == 1) {
delete_at_beginning();

```

```

return;
}
struct node *temp = head;
int i = 1;
while (i < location - 1 && temp != NULL) {
temp = temp->next;
i++;
}
if (temp == NULL) {
printf("The specified location is out of bounds.\n");
return;
}
struct node *to_delete = temp->next;
temp->next = temp->next->next;
free(to_delete);
}
void display() {
struct node *temp = head;
while (temp != NULL) {
printf("%d ", temp->data);
temp = temp->next;
}
printf("\n");
}
int main() {
int choice;
int data, location;
while (1) {
printf("\nLinked List Operations\n");
printf("1. Insert at beginning\n");
printf("2. Insert at end\n");
printf("3. Insert at specified location\n");
printf("4. Delete at beginning\n");
printf("5. Delete at end\n");
printf("6. Delete at specified location\n");
printf("7. Display\n");
printf("8. Exit\n");
printf("\nEnter your choice: \n");
scanf("%d", &choice);
switch (choice) {
case 1:
printf("Enter the data to be inserted:\n ");
scanf("%d", &data);
insert_at_beginning(data);
break;

```

```

case 2:
printf("Enter the data to be inserted:\n ");
scanf("%d", &data);
insert_at_end(data);
break;
case 3:
printf("Enter the data to be inserted:\n ");
scanf("%d", &data);
printf("Enter the location of insertion:\n ");
scanf("%d", &location);
insert_at_specified_location(data,location);
break;
case 4:
delete_at_beginning();
break;
case 5: delete_at_end();
break;
case 6:printf("Enter the location of deletion: \n");
scanf("%d", &location);
delete_at_specified_location(location);
break;
case 7:display();
break;
case 8: printf("Exiting\n");
return;
break;
default: printf("Invalid choice\n");
}
}
}

```

Output:

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

1

Enter the data to be inserted:

1

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

2

Enter the data to be inserted:

2

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

3

Enter the data to be inserted:

3

Enter the location of insertion:

2

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

7

1 3 2

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning

5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

4

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

7

3 2

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

1

Enter the data to be inserted:

1

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

7

1 3 2

Linked List Operations

1. Insert at beginning

2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

6

Enter the location of deletion:

2

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

7

1 2

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

5

Linked List Operations

1. Insert at beginning
2. Insert at end
3. Insert at specified location
4. Delete at beginning
5. Delete at end
6. Delete at specified location
7. Display
8. Exit

Enter your choice:

7

1

Linked List Operations

1. Insert at beginning
 2. Insert at end
 3. Insert at specified location
 4. Delete at beginning
 5. Delete at end
 6. Delete at specified location
 7. Display
 8. Exit
- Enter your choice:

8

Exiting

Assignment – 5

Aim: Implementation of evaluation of postfix expression

Code:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 100
// Stack implementation
int stack[MAX_SIZE];
int top = -1;
void push(int item) {
    if (top >= MAX_SIZE - 1) {
        printf("Stack Overflow\n");
        return;
    }
    top++;
    stack[top] = item;
}
int pop() {
    if (top < 0) {
        printf("Stack Underflow\n");
        return -1;
    }
    int item = stack[top];
    top--;
    return item;
}
int is_operator(char symbol) {
    if (symbol == '+' || symbol == '-' || symbol == '*' || symbol == '/') {
```

```

return 1;
}
return 0;
}
int evaluate(char* expression) {
int i = 0;
char symbol = expression[i];
int operand1, operand2, result;

while (symbol != '\0') {
if (symbol >= '0' && symbol <= '9') {
int num = symbol - '0';
push(num);
}
else if (is_operator(symbol)) {
operand2 = pop();
operand1 = pop();
switch(symbol) {
case '+': result = operand1 + operand2; break;
case '-': result = operand1 - operand2; break;
case '*': result = operand1 * operand2; break;
case '/': result = operand1 / operand2; break;
}
push(result);
}
i++;
symbol = expression[i];
}
result = pop();
return result;
}

int main() {
char expression[] = "5 6 7 + * 8 -";
int result = evaluate(expression);
printf("Result= %d\n", result);
return 0;
}

```

Output:

The output of the above program will be:

Result: 57

Algorithm:

1. Start scanning the expression from left to right.
2. Push operand 5 onto the stack.

3. Push operand 6 onto the stack.
4. Push operand 7 onto the stack.
5. Pop operands 7 and 6 from the stack, perform addition, and push the result (13) back onto the stack.
6. Pop operands 13 and 5 from the stack, perform multiplication, and push the result (65) back onto the stack.
7. Push operand 8 onto the stack.
8. Pop operands 8 and 65 from the stack, perform subtraction, and push the result (57) back onto the stack.
9. The final result is 57.

Assignment – 6

Aim: Implementation of Stack and Queue using Singly Linked List

Code (Stack):

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int info;
    struct node *ptr;
}*top,*top1,*temp;
int count = 0;
void push(int data) {
    if (top == NULL)
    {
        top =(struct node *)malloc(1*sizeof(struct node));
        top->ptr = NULL;
        top->info = data;
    }
    else
    {
        temp =(struct node *)malloc(1*sizeof(struct node));
        temp->ptr = top;
        temp->info = data;
        top = temp;
    }
    count++;
    printf("Node is Inserted\n\n");
}
int pop() {
    top1 = top;
    if (top1 == NULL)
    {
```

```

printf("\nStack Underflow\n");
return -1;
}
else
top1 = top1->ptr;
int popped = top->info;
free(top);
top = top1;
count--;
return popped;
}
void display()
{
top1 = top;
if (top1 == NULL)
{
printf("\nStack Underflow\n");
return;
}

```

```

printf("The stack is \n");
while (top1 != NULL)
{
printf("%d--->", top1->info);
top1 = top1->ptr;
}
printf("NULL\n\n");
}
int main() {
int choice, value;
printf("\nImplementation of Stack using Linked List\n");
while (1) {
printf("\n1. Push\n2. Pop\n3. Display\n4. Exit\n");
printf("\nEnter your choice : ");
scanf("%d", &choice);
switch (choice) {
case 1:
printf("\nEnter the value to insert: ");
scanf("%d", &value);
push(value);
break;
case 2:
printf("Popped element is :%d\n", pop());
break;
case 3:

```

```
display();
break;
case 4:
exit(0);
break;
default:
printf("\n Wrong Choice\n");
}
}
}
```

Output:

Implementation of Stack using Linked List

1. Push

2. Pop

3. Display

4. Exit

Enter your choice : 1

Enter the value to insert: 11

Node is Inserted

1. Push

2. Pop

3. Display

4. Exit

Enter your choice : 1

Enter the value to insert: 12

Node is Inserted

1. Push

2. Pop

3. Display

4. Exit

Enter your choice : 3

The stack is

12--->11--->NULL

1. Push

2. Pop

3. Display

4. Exit

Enter your choice : 2

Popped element is :12

1. Push

2. Pop

3. Display

4. Exit

Enter your choice : 3

The stack is

11--->NULL

1. Push
2. Pop
3. Display
4. Exit

Enter your choice : 4

Code (Queue):

```
#include < stdio.h >
#include < stdlib.h >
struct node {
    int data;
    struct node * next;
};
struct node * front = NULL;
struct node * rear = NULL;
void enqueue(int value) {
    struct node * ptr;
    ptr = (struct node * ) malloc(sizeof(struct node));
    ptr -> data = value;
    ptr -> next = NULL;
    if ((front == NULL) && (rear == NULL)) {
        front = rear = ptr;
    } else {
        rear -> next = ptr;
        rear = ptr;
    }
    printf("Node is Inserted\n\n");
}
int dequeue() {
    if (front == NULL) {
        printf("\nUnderflow\n");
        return -1;
    } else {
        struct node * temp = front;
        int temp_data = front -> data;
        front = front -> next;
        free(temp);
        return temp_data;
    }
}
void display() {
    struct node * temp;
    if ((front == NULL) && (rear == NULL)) {
```

```

printf("\nQueue is Empty\n");
} else {
printf("The queue is \n");
temp = front;
while (temp) {
printf("%d--->", temp -> data);
temp = temp -> next;
}
printf("NULL\n\n");
}
}

int main() {
int choice, value;
printf("\n Implementation of Queue using Linked List\n");
while (choice != 4) {
printf("1.Enqueue\n2.Dequeue\n3.Display\n4.Exit\n");
printf("\nEnter your choice : ");
scanf("%d", & choice);
switch (choice) {
case 1:
printf("\nEnter the value to insert: ");
scanf("%d", & value);
enqueue(value);
break;
case 2:
printf("Popped element is :%d\n", dequeue());
break;
case 3:
display();
break;
case 4:
exit(0);
break;
default:
printf("\n Wrong Choice\n");
}
}
return 0;
}

```

Output:

Implementation of Queue using Linked List

1. Enqueue
2. Dequeue
3. Display

4. Exit

Enter your choice: 1

Enter the value to insert: 12

Node is Inserted

1.Enqueue

2.Dequeue

3.Display

4.Exit

Enter your choice: 1

Enter the value to insert: 45

Node is Inserted

1.Enqueue

2.Dequeue

3.Display

4.Exit

Enter your choice: 1

Enter the value to insert: 56

Node is Inserted

1.Enqueue

2.Dequeue

3.Display

4.Exit

Enter your choice : 3

The queue is

12--->45--->56--->NULL

Assignment – 7

Aim: Implementation of Doubly Linked List

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct node
{
    struct node *prev;
    struct node *next;
    int data;
};
struct node *head;
void insertion_beginning();
void insertion_last();
void insertion_specified();
void deletion_beginning();
```

```

void deletion_last();
void deletion_specified();
void display();
void search();
int main()
{
int choice = 0;
while (choice != 9)
{
printf("\nChoose one option from the following list ...\n");
printf("\n===== \n");
printf("\n1.Insert in beginning\n2.Insert at last\n3.Insert at any random
location\n4.Delete from Beginning\n 5.Delete from last \n6.Delete the node
after the
given data\n7.Search\n8.Show\n9.Exit\n ");
printf("\n Enter your choice?\n");
scanf("\n%d", &choice);
switch (choice)
{
case 1:
insertion_beginning();
break;
case 2:
insertion_last();
break;
case 3:
insertion_specified();
break;
case 4:
deletion_beginning();
break;
case 5:
deletion_last();
break;
case 6:
deletion_specified();
break;
case 7:
search();
break;
case 8:
display();
break;
case 9:
exit(0);

```

```

break;
default:
printf("Please enter valid choice..");
}
}
return 0;
}
void insertion_beginning()
{
struct node *ptr;
int item;
ptr = (struct node *)malloc(sizeof(struct node));
if (ptr == NULL)
{
printf("\nOVERFLOW");
}
else
{
printf("\nEnter Item value");
scanf("%d", &item);
if (head == NULL)
{
ptr->next = NULL;
ptr->prev = NULL;
ptr->data = item;
head = ptr;
}
else
{
ptr->data = item;
ptr->prev = NULL;
ptr->next = head;
head->prev = ptr;
head = ptr;
}
printf("\nNode inserted\n");
}
}
void insertion_last()
{
struct node *ptr, *temp;
int item;
ptr = (struct node *)malloc(sizeof(struct node));
if (ptr == NULL)
{

```

```

printf("\nOVERFLOW");
}
else
{
printf("\nEnter value");
scanf("%d", &item);
ptr->data = item;
if (head == NULL)
{
ptr->next = NULL;
ptr->prev = NULL;
head = ptr;
}
else
{
temp = head;
while (temp->next != NULL)
{
temp = temp->next;
}
temp->next = ptr;
ptr->prev = temp;
ptr->next = NULL;
}
}
printf("\nnode inserted\n");
}
void insertion_specified()
{
struct node *ptr, *temp;
int item, loc, i;
ptr = (struct node *)malloc(sizeof(struct node));
if (ptr == NULL)
{
printf("\n OVERFLOW");
}
else
{
temp = head;
printf("Enter the location");
scanf("%d", &loc);
for (i = 0; i < loc; i++)
{
temp = temp->next;
if (temp == NULL)

```

```

{
printf("\n There are less than %d elements", loc);
return;
}
}

```

```

printf("Enter value");
scanf("%d", &item);
ptr->data = item;
ptr->next = temp->next;
ptr->prev = temp;
temp->next = ptr;
temp->next->prev = ptr;
printf("\nnode inserted\n");
}
}

```

```

void deletion_beginning()
{
struct node *ptr;
if (head == NULL)
{
printf("\n UNDERFLOW");
}
else if (head->next == NULL)
{
head = NULL;
free(head);
printf("\nnode deleted\n");
}
else
{
ptr = head;
head = head->next;
head->prev = NULL;
free(ptr);
printf("\nnode deleted\n");
}
}

```

```

void deletion_last()
{
struct node *ptr;
if (head == NULL)
{
printf("\n UNDERFLOW");
}
else if (head->next == NULL)

```

```

{
head = NULL;
free(head);
printf("\nnode deleted\n");
}
else
{
ptr = head;
if (ptr->next != NULL)
{
ptr = ptr->next;
}
ptr->prev->next = NULL;
free(ptr);
printf("\nnode deleted\n");
}
}
void deletion_specified()
{
struct node *ptr, *temp;
int val;
printf("\n Enter the data after which the node is to be deleted : ");
scanf("%d", &val);
ptr = head;
while (ptr->data != val)
ptr = ptr->next;
if (ptr->next == NULL)
{
printf("\nCan't delete\n");
}
else if (ptr->next->next == NULL)
{
ptr->next = NULL;
}
else
{
temp = ptr->next;
ptr->next = temp->next;
temp->next->prev = ptr;
free(temp);
printf("\nnode deleted\n");
}
}
void display()
{

```

```

struct node *ptr;
printf("\n printing values...\n");
ptr = head;
while (ptr != NULL)
{
printf("%d\n", ptr->data);
ptr = ptr->next;
}
}
void search()
{
struct node *ptr;
int item, i = 0, flag;
ptr = head;
if (ptr == NULL)
{
printf("\nEmpty List\n");
}
else
{
printf("\nEnter item which you want to search?\n");
scanf("%d", &item);
while (ptr != NULL)
{
if (ptr->data == item)
{
printf("\nitem found at location %d ", i + 1);
flag = 0;
break;
}
else
{
flag = 1;
}
i++;
ptr = ptr->next;
}
if (flag == 1)
{
printf("\n Item not found\n");
}
}
}

```

Output:

Choose one option from the following list ...

=====

- 1.Insert in beginning
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

1

Enter Item value10

Node inserted

Choose one option from the following list ...

=====

- 1.Insert in beginning
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

2

Enter value11

node inserted

Choose one option from the following list ...

=====

- 1.Insert in beginning
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

2

Enter value13

node inserted

Choose one option from the following list ...

=====

- 1.Insert in beginning
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

8

printing values...

10

11

13

Choose one option from the following list ...

=====

- 1.Insert in beginning
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

Assignment - 8

Aim: Implementation of Binary Search Tree

Code:

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
    int key;
    struct node*left,*right;
};
struct node *newNode(int item){
    struct node *temp=(struct node*)malloc(sizeof(struct node));
```

```

temp->key=item;
temp->left=temp->right=NULL;
return temp;
};
void inorder(struct node* root){
if(root!=NULL){
inorder(root->left);
printf("%d->",root->key);
inorder(root->right);
}
}
struct node*insert(struct node*node,int key){
if(node==NULL)
return newNode(key);
if(key<node->key)
node->left=insert(node->left,key);
else
node->right=insert(node->right,key);
return node;
}
struct node*minValueNode(struct node*node){
struct node*current=node;
while(current && current->left!=NULL)
current=current->left;
return current;
}
struct node*deleteNode(struct node*root,int key){
if(root==NULL)
return root;
if(key<root->key)
root->left=deleteNode(root->left,key);
else
if(key>root->key)
root->right=deleteNode(root->right,key);
else{
if(root->left==NULL){
struct node* temp=root->right;
free(root);
return temp;
}
else if(root->right==NULL){
struct node*temp=root->left;
free(root);
return temp;
}
}
}

```

```

struct node*temp=minValueNode(root->right);
root->key=temp->key;
root->right=deleteNode(root->right,temp->key);
}
return root;
};
int main()
{
struct node*root=NULL;
root=insert(root,8);
root=insert(root,3);
root=insert(root,1);
root=insert(root,6);
root=insert(root,7);
root=insert(root,10);
root=insert(root,14);
root=insert(root,4);
printf("Inorder traversal:");
inorder(root);
printf("\nAfter deleting 10\n");
root=deleteNode(root,10);
printf("inorder traversal: ");
inorder(root);
}

```

Output:

Inorder traversal:1->3->4->6->7->8->10->14->

After deleting 10

inorder traversal: 1->3->4->6->7->8->14->

Process returned 0 (0x0) execution time : 0.031 s

Press any key to continue.

Assignment - 9

Aim: Implementation of Adjacency Matrix Creation

Code:

```

#include<stdio.h>
#include<conio.h>
void main()
{
int a[10][10] = {0};
int node,edge,start,end,i,j;
printf("Enter the number of nodes/vertices : ");
scanf("%d",&node);

```

```

printf("\n");
edge = (node*(node-1)/2);
//edge = (node*(node-1));
for(i=0;i<edge;i++){
printf("Enter the start point : ");
scanf("%d",&start);
printf("Enter the end point : ");
scanf("%d",&end);
printf("\n");
a[start][end]=1;
a[end][start]=1;
}
for(i=0;i<node;i++){
for(j=0;j<node;j++){
printf("%d\t",a[i][j]);
}
printf("\n");
}
}

```

Output:

```

Enter the number of nodes/vertices : 4
Enter the start point : 2
Enter the end point : 4
Enter the start point : 3
Enter the end point : 5
Enter the start point : 4
Enter the end point : 6
Enter the start point : 5
Enter the end point : 7
Enter the start point : 2
Enter the end point : 6
Enter the start point : 3
Enter the end point : 5
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0

```

• DFS(Depth- First- Search) Algorithm

```

DFS(graph, start_node):
create an empty stack
push start_node onto the stack

```

```

while stack is not empty:
current_node = pop stack
if current_node has not been visited:
mark current_node as visited
process current_node (e.g., print it)
for each neighbor in graph.adjacent_nodes(current_node):
if neighbor has not been visited:
push neighbor onto the stack.

```

BFS (Breadth-First-Search) Algorithm

```

BFS(graph, start_node):
create an empty queue
enqueue start_node into the queue
mark start_node as visited
while queue is not empty:
current_node = dequeue from the queue
process current_node (e.g., print it)
for each neighbor in graph.adjacent_nodes(current_node):
if neighbor has not been visited:
mark neighbor as visited
enqueue neighbor into the queue.

```

Assignment - 10

Aim: Implementation of Hashing Function with different Collision Tech

Consider a hash table of size 10
Keys: 72, 24, 36, 24, 63, 81, 91,100

Linear probing:

$$h(x,i) = (h(x) + i) \bmod 10$$

Insert 72 (Step 1).

$$(72,0) = (72 + 0) \bmod 10 \\ = 2$$

Insert 27 (Step 2)

$$(27, 0) = (27 + 0) \bmod 10 \\ = 7$$

Insert 36 (Step 3)

$$(36, 0) = (36 + 0) \bmod 10$$

0	100
1	81
2	72
3	63
4	24
5	91
6	36
7	27
8	

=6

Insert 24 (Step 4)

$$(24, 0) = (24 + 0) \bmod 10$$

=4

Insert 63 (Step 5)

$$(63, 0) = (63 + 0) \bmod 10$$

=3

Insert 81 (Step 6)

$$(81, 0) = (81 + 0) \bmod 10$$

=1

Insert 91 (Step 7)

$$(91, 0) = (91 + 0) \bmod 10$$

=1 (Collision)

Step 8;

$$(91, 2) = (91 + 1) \bmod 10$$

=27

Step 9:

$$(91, 2) = (91 + 2) \bmod 10$$

=3

Step

10:

$$(91, 3) = (91 + 3) \bmod 10$$

=4

Step

11:

$$(91, 4) = (91 + 4) \bmod 10$$

=5 (Collision)

Insert 100 (Step 12)

$$(100, 0) = (100 + 0) \bmod 10$$

=0

Quadratic Probing:

$$h(x, i) = (h(x) + i^2) \bmod 10$$

Insert 72 (Step 1)

$$(72, 0) = (72 + 0) \bmod 10 = 2$$

$$\text{Insert } 27 \text{ } i=0, h(27,0) = 7$$

$$\text{Insert } 36 \text{ } i=0, h(36,0) = 6$$

$$\text{Insert } 24 \text{ } i=0, h(24,0) = 4$$

$$\text{Insert } 63 \text{ } i=0, h(63,0) = 3$$

$$\text{Insert } 81 \text{ } i=0, h(81,0) = 1$$

$$\text{Insert } 91 \text{ } i=0, h(91,0) = 1 - \text{Collision } i=0, h(91,1) = 2 - \text{Collision } i=0, h(91,2) = 5$$

$$\text{Insert } 100 \text{ } i=0, h(100,0) = 0$$

0	100
1	81
2	72
3	63
4	24
5	91
6	36
7	27
8	

Double Hashing:

$$h(x,p) = (h_1(x,i) + p \cdot h_2(x,i)) \bmod 10$$

$$h_2(x) = \text{prime} - (\text{key} \% \text{prime}) \ \& \ \text{prime} < \text{tables}$$

$$\text{hash } 1 (72) = 72 \% 10 = 2$$

$$\text{hash } 1 (27) = 27 \% 10 = 7$$

$$\text{hash } 1 (36) = 36 \% 10 = 6$$

$$\text{hash } 1 (24) = 24 \% 10 = 4$$

$$\text{hash } 1 (63) = 63 \% 10 = 3$$

$$\text{hash } 1 (81) = 81 \% 10 = 1$$

$$\text{hash } 1 (91) = 91 \% 10 = 1$$

$$\text{hash } 2 (91) = 7 - (91 \% 7)$$

$$= 7$$

$$[\text{hash}(91) + 1 \cdot \text{hash}_2(91)] \% 10$$

$$= [1 + 7] \% 10$$

$$= 8$$

$$\text{Hash } 1 (100) = 100 \% 10 = 0$$

$$\text{Prime} = 7$$

0	100
1	81
2	72
3	63
4	24
5	91
6	36
7	27
8	